

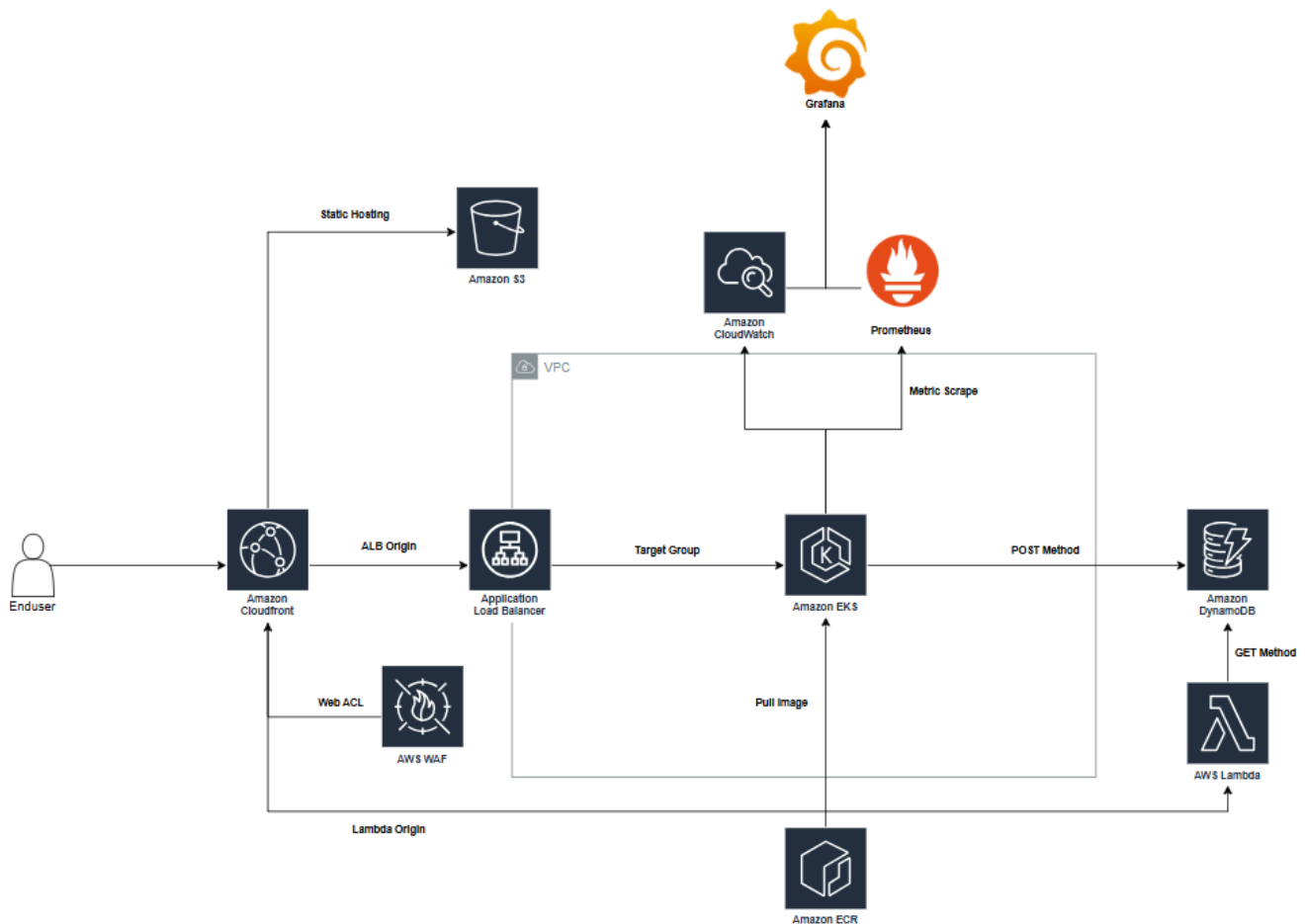
2026년도 전국대회 1과제

직 종 명	클라우드컴퓨팅	과제명	Solution Architecture	과제번호	제1과제
경기시간	4시간	비번호		심사위원 확 인	(인)

1. 요구사항

당신은 skills.inc에서 클라우드 솔루션을 활용해 애플리케이션을 인프라를 구성하도록 합니다. AWS 서비스를 이용해 애플리케이션과 컨테이너 인프라를 효율적으로 관리하도록 하며, 아래 다이어그램과 주어진 요구사항과 클라우드의 설계원칙을 준수해 인프라를 구축하도록 합니다.

다이어그램



Software Stack

AWS	개발언어/프레임워크
- VPC - EC2 - ELB - CloudFront - WAF - S3 - EKS - ECR - DynamoDB - KMS - Lambda	- golang/gin - docker

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하십시오.

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방하여 주시기 바랍니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 문제에 제시된 괄호 <>는 변수를 뜻함으로 선수가 적절히 변경하여 사용해야 합니다.
- 6) Security Group의 80/443 outbound는 anyopen하여 사용할 수 있도록 합니다.
- 7) 과제 종료 전 실행 중인 테스트 및 부하를 중지하여 서버에 문제가 없도록 해야 합니다.
- 8) Tag 정보가 맞지 않거나 권한 문제가 생겨 채점이 불가하지 않도록 주의합니다.
- 9) Region 설정이 필요한 리소스는 모두 서울 지역(ap-northeast-2)에 구성합니다.
- 10) CMK 구성의 경우 최소 권한으로 정책을 설정하기 위해 root와 "kms": "*"정책을 금지합니다.

CmxGraphModel%3E%3Croot%3E%3CmxCell%20id%3D%220%22%2F%3E%3CmxCell%20id%3D%221%22%20parent%3D%220%22%2F%3E%3CmxCell%20id%3D%222%22%20value%3D%22%22%20style%3D%22sketch%3D0%3BoutlineConnect%3D0%3BfontColor%3D%232323%3BgradientColor%3Dnone%3BfillColor%3D%23ED7100%3BstrokeColor%3Dnone%3Bdashed%3D0%3BverticalLabelPosition%3Dbottom%3BverticalAlign%3Dtop%3Balign%3Dcenter%3Bhtml%3D1%3BfontSize%3D12%3BfontStyle%3D0%3Baspect%3Dfixed%3BpointerEvents%3D1%3Bshape%3Dmxgraph.aws4.ecs_task%3B%22%20vertex%3D%221%22%20parent%3D%221%22%3E%3CmxGeometry%20x%3D%22414%22%20y%3D%22471%22%20width%3D%2237%22%20height%3D%2248%22%20as%3D%22geometry%22%2F%3E%3C%2FmxCell%3E%3C%2Froot%3E%3C%2FmxGraphModel%3E

3. Networking

AWS 내에서 가상 사설 Network를 구축할 수 있도록 아래의 설명과 Reference01의 표를 참고하여 VPC를 구성하도록 합니다. 퍼블릭과 프라이빗 서브넷 모두 고가용성을 보장하여 생성합니다. 퍼블릭 서브넷의 경우 직접적으로 외부에 통신이 되어야하지만, 프라이빗 서브넷의 경우 NAT Gateway를 통해 외부와 통신되도록 구성합니다.

4. Application

Book 애플리케이션은 콘서트 예약 정보를 받아 데이터베이스에 저장하는 POST API를 제공합니다. Book 어플리케이션은 구동을 위해서 AWS_REGION 과 TABLE_NAME이라는 환경 변수를 필요로 합니다. 정상 구동 시 8080 포트가 바인딩 되며 /health 호출 시 200 OK 를 반환합니다. 애플리케이션 구동 시 필요한 환경변수를 Dockerfile, deployment등에 직접적으로 적지 않고, configmap으로 관리하여 애플리케이션 구동할 수 있도록 합니다.

– Configmap Name : book-config

API	Request	Response	Description
POST /v1/book	application/json – client_id: Client ID (String) – username: 구매자 이름 (String) – email: 구매자 이메일 (String) – concert_name: 콘서트 (String) Sample: { "client_id": "C001", "username": "Alice", "email": "kim@example.com", "concert_name": "Seoul2025" }	Code 200 – booking_id: 유니크 예약 ID (String) Sample: { "booking_id": "C2011YY" }	HTTP 요청의 Body에 있는 필드 값과 created_at 필드가 DynamoDB 테이블에 저장됩니다

5. Database

애플리케이션과 호환되는 NoSQL 데이터베이스인 DynamoDB를 구성합니다. 비용 효율성을 위해 PAY_PER_REQUEST 모드로 생성하며, 보안 강화를 위해 KMS CMK로 데이터를 암호화합니다. 실수로 인한 데이터 손실을 방지하기 위해 삭제 방지 기능을 활성화하고, EKS Pod에는 데이터를 삽입할 수 있는 권한, Lambda에는 저장된 데이터를 조회할 수 있는 권한만을 테이블 수준에서 부여하여 최소 권한 원칙을 준수합니다. 데이터 손실을 방지하기 위해 PITR을 활성화하여 데이터를 최장 기간 복구할 수 있도록 설정하고, booking_id를 이용한 효율적인 조회를 위해 GSI를 구성하도록 합니다.

– Table Name : wsc2026-book-table

– PK : client_id

– CMK Name : wsc2026-db-kms

6. Container Registry

애플리케이션이 포함된 컨테이너 이미지를 ECR로 올려서 관리하도록 합니다. Private registry를 구성하고 보안 구성들을 진행합니다. 업로드 시 스캐닝이 되어야 하며 이미지에 취약점이 존재하면 안 됩니다. 또한 같은 이름의 이미지 태그가 있어도 업로드가 되도록 구성하되, v1.0.0, v1.0.1 등 v1 버전을 명시하는 이미지 태그는 예외입니다. 보안을 위해 KMS CMK로 암호화하도록 구성합니다. ECR에는 v1.0.0 이미지를 제외한 이미지가 존재하면 안 됩니다.

– ECR name : wsc2026-book-ecr

– CMK Name :wsc2026-ecr-kms

7. Container Orchestration

지급받은 애플리케이션을 컨테이너 오케스트레이션 환경에서 운영하기 위해 Amazon AWS의 EKS 서비스를 사용합니다. 해당 EKS 클러스터들에서 발생하는 모든 Control Plane 로그는 CloudWatch에 저장하여 가시성을 확보하고, 보안성을 극대화하기 위해 KMS CMK를 이용한 암호화를 적용합니다. 또한, 모든 클러스터는 외부 노출 없이 내부에서만 통신이 가능하도록 Fully Private으로 구축하며, 보안 그룹의 인바운드 규칙에서는 Any IP 허용을 철저히 배제하며, EKS와 노드 그룹에 부여되는 IAM Role은 Administrator와 같은 과도한 권한 대신 실제 운영에 필요한 최소 권한만을 적용하여 사용합니다. Kubernetes 내부에서 사용하는 Domain을 기존 *.cluster.local에서 *.wsc2026.skills.local로 변경합니다.

- Cluster Name : wsc2026-eks-cluster
- Cluster Version : 1.35
- CMK Name : wsc2026-eks-kms

Addon NodeGroup

데몬셋을 제외한 애플리케이션 이외의 Addon 또는 시스템 구성 요소들은 별도의 Addon NodeGroup에서 운영합니다. 해당 노드에는 { wsc2026/node: addon } 라벨이 설정되어 있어야 합니다.

- NodeGroup Name: wsc2026-addon-nodegroup
- Node Instance Name: wsc2026-addon-node
- Node Instance Type: t3.medium

Application NodeGroup

애플리케이션이 구동되는 NodeGroup입니다. 데몬셋을 제외하고 해당 노드 그룹에선 애플리케이션 리소스만 존재해야 합니다. 해당 노드에는 { wsc2026/node: application } 라벨이 설정되어 있어야 합니다.

- NodeGroup Name: wsc2026-workload-ng
- Node Instance Name: wsc2026-workload-node
- Node Instance Type: t3.medium

8. Deployment

Book 애플리케이션을 wsc2026 Namespace에 Deployment로 배포합니다.고가용성 확보를 위해 최소 2대의 Pod를 가용 영역 간 균등 분산하여 특정 영역에 편중되지 않도록 하고, 노드 드레인이나 클러스터 업그레이드 시에도 최소 1대의 Pod가 가용 상태를 유지하도록 구성합니다. readinessProbe, livenessProbe, startupProbe를 / health:8080으로 설정하며, Pod Identity로 DynamoDB 최소 접근 권한을 부여하고, 서비스는 동일 가용 영역 내 Pod로 우선 라우팅하여 Cross-AZ 트래픽 비용을 절감합니다.

- Deployment Name : wsc2026-book-deploy
- Service Name : wsc2026-book-svc
- ingress Name : wsc2026-book-ingress
- Pod Identity role Name : wsc2026-book-pod-role
- Service Account Name : wsc2026-book-sa
- PDB Name : wsc2026-book-pdb
- CPU/Memory
 - Request, Limits : .25 vCPU/ .5GiB

9. S3 Bucket

Static Page Hosting을 위해 S3를 구성하고 Cloudfront를 이용해서 호스팅을 하도록 합니다. S3에 지급받은 배포파일들을 static/에 전부 업로드해두도록 하며, Private 환경으로 구성하고 오직 Cloudfront에서만 접근 가능하도록 설정합니다. 버킷과 객체 모두 SSE-KMS 형태로 KMS CMK로 암호화하고 버킷 키를 활성화합니다.

- S3 Bucket Name : wsc2026-static-⟨임의의 영문 4자리>-⟨본인 비번호>-bucket
- CMK Name : wsc2026-bucket-kms

10. Lambda

지급받은 book 애플리케이션에는 GET 메서드가 존재하지 않기에 Lambda를 이용해 GET 메서드를 구현하도록 합니다. /v1/book이라는 경로를 통해 GET이 되도록 하며, booking_id 쿼리스트링을 이용해 Dynamodb에 저장된 애플리케이션 데이터를 조회할 수 있도록 합니다. 람다 코드는 KMS CMK로 암호화되어야 합니다. Dynamodb에 데이터를 조회하기 위해 필요한 정보를 담은 환경변수인 TABLE_NAME은 Lambda Environment variables로 관리하며, 보안을 위해 환경변수는 전송 중과 저장 중일 때 CMK로 암호화하도록 합니다. 과도한 권한 사용을 배제하기 위해 최소 권한을 가진 IAM role을 생성하여 사용하도록 합니다.

- Lambda Function Name : wsc2026-book-get-function
- Runtime : Python 3.12
- IAM policy : wsc2026-book-function-policy
- IAM role : wsc2026-book-function-role
- CMK Name : wsc2026-function-kms

API	Request	Response	Description
GET /v1/book	- booking_id: Booking ID (String) Sample: ?booking_id=Booking ID	<pre>{ "client_id" : "Client ID", "username" : "Alice", "email": "kim@example.com" , "concert_name" : "Seoul2025", "created_at" : "2026-04-22 14:33:38 KS" }</pre>	GET 요청 시 반환되는 컬럼들의 순서가 Response와 정확히 일치해야하며, created_at의 경우 Response처럼 정해진 포맷으로 변환되어야 한다.

11. Observability

EKS 노드 그룹과 애플리케이션의 효율적인 관리 및 모니터링을 위해 Prometheus, Alertmanager를 활용한 오픈서버빌리티 환경을 Addon Node에 구축합니다. Prometheus를 통해 인프라와 애플리케이션의 메트릭을 수집하며, 노드 수준의 메트릭 수집을 위해 Node Exporter를 DaemonSet으로 배포하고 데이터 보존 기간은 7일로 설정합니다. 시스템 이상 징후 발생 시 즉각적인 대응이 가능하도록 Prometheus Alertmanager를 통해 알람 체계를 구성합니다. 애플리케이션에서 발생하는 액세스 로그는 Fluent Bit을 통해 수집하여 CloudWatch Logs로 전송합니다. Fluent Bit은 DaemonSet으로 배포하며, health 로그는 제외하고 실제 API 요청 로그만 수집합니다. 수집된 로그는 파싱하며 Reference02의 형태를 참고하여 Grafana 대시보드에서 조회 가능하도록 구조화합니다. 최종적으로 Grafana를 Prometheus, Alertmanager, CloudWatch 데이터 소스와 연동하여 통합 대시보드를 구성하며, Service 타입을 LB로 지정하여 통해 외부 브라우저에서 접근이 가능하도록 설정합니다. Grafana의 CloudWatch Logs 데이터 소스를 통해 애플리케이션 로그를 조회할 수 있도록 합니다. 대시보드 상단에는 노드 그룹 및 네임스페이스별로 데이터를 필터링할 수 있도록 구성하고, 임계치 기반 색상 변환을 적용하여 CPU 80% 이상은 빨간색, 60~80%는 노란색, 60% 미만은 초록색으로 표시하며 Pod 재시작 횟수가 1회 이상 발생할 경우 경고 색상을 점등합니다. 자세한 내용은 Reference02를 참고해 구성하도록 합니다.

- Grafana Dashboard Name : wsc2026-grafana-dashboard
- Grafana Admin Password : Skills\$#s@!
- Namespace : observability

12. Load Balancer

EKS에서 운영되는 애플리케이션의 엔드포인트 제공과 부하 분산을 위해 EKS에서 AWS Load Balancer Controller를 이용해 ALB를 구성합니다. ALB는 외부에 직접 노출하는 internet-facing으로 구성하며, CloudFront를 통해서만 접근할 수 있도록 합니다. 잘못된 경로의 요청은 403을 반환하도록 구성합니다.

- ALB Name : wsc2026-app-alb
- ALB scheme : internet-facing

- SG Name : wsc2026-app-alb-sg

13. CDN

사용자가 애플리케이션에 접근하기 위한 단일 진입점으로 Amazon CloudFront를 구성합니다. CloudFront는 정적 페이지 호스팅, 애플리케이션 API, 데이터 조회 API를 각각 다른 Origin으로 연결하여 하나의 도메인에서 모든 서비스를 제공합니다. 사전에 구성한 S3를 CloudFront에서 불러와 정적 페이지 호스팅을 하도록 하며, 사용자가 루트 경로로 접근 시 정적 페이지가 표시되도록 합니다. EKS에서 운영되는 Book 애플리케이션의 POST API는 CloudFront의 /booking 경로로 사용자에게 노출합니다. CloudFront는 해당 경로의 요청을 ALB Origin으로 전달하여 애플리케이션을 사용할 수 있도록 합니다. DynamoDB에 저장된 데이터를 조회하는 GET API는 Lambda로 구현하며 /v1/book 경로를 사용하고 Lambda Function URL을 CloudFront의 별도 Origin으로 연결합니다. CloudFront에 AWS WAF WebACL을 연결하여 SQL Injection 및 XSS 공격에 대한 룰셋을 적용하고, Rate Limiting 룰을 추가하여 1분간 특정 IP로부터의 200개 이상의 과도한 요청을 차단합니다. 캐싱의 경우 S3는 활성화하고 ALB와 Lambda는 비활성화하도록 합니다.

- CDN Name : wsc2026-cdn

- WAF Name : wsc2026-waf

Reference01

- VPC -

Name	CIDR
wsc2026- skills- vpc	192.168.0.0/16

- subnets -

Name	CIDR	RTB	Internet
wsc2026- skills- hub- sub- a	192.168.1.0/24	wsc2026- skills- hub- rtb	wsc2026- skills- igw
wsc2026- skills- hub- sub- b	192.168.10.0/24	wsc2026- skills- hub- rtb	wsc2026- skills- igw
wsc2026- skills- app- sub- a	192.168.2.0/24	wsc2026- skills- app- rtb- a	wsc2026- skills- nat- a
wsc2026- skills- app- sub- b	192.168.20.0/24	wsc2026- skills- app- rtb- b	wsc2026- skills- nat- b

Reference02

- Prometheus Alert -

Alerts Name	발생 원인
PodHighCPU	Pod CPU 사용률 80%를 3분 이상 초과하는 경우
PodHighMemory	Pod 메모리 사용률 90%를 3분 이상 초과하는 경우
PodNotReady	Pod가 CrashLoopBackOff 상태이거나 3분 이상 Ready가 아닌 경우
HighErrorRate	4xx나 5xx 에러 비율이 전체 요청의 5%를 1분 이상 초과하는 경우
HighLatency	평균 응답 시간이 3초를 1분 이상 초과하는 경우
PodCrashLooping	wsc2026 NS Pod의 컨테이너 재시작 횟수가 3회를 초과한 상태가 3분 이상 지속되는 경우

- Grafana Dashboard -

Low	Panel
Node	All Node CPU / All Node Memory / All Available Nodes
Pod	All Pod CPU / All Pod Memory / All Pending Pods / All Pod restarts
Application Pod	App Pod CPU / App Pod Memory / App Running / App restarts / App Pending
Application Traffic	App Request Count / App Response Time / App Status Code / App Application Logs
Alert	Alertmanager 알림 현황

- Log Format -

INFO {"level":"INFO","path":"/v1/book","status":"200","duration":"75.959276ms","method":"POST"}

WARN {"level":"WARN","path":"/v1/book/999","status":"404","duration":"12.345678ms","method":"GET"}

ERROR {"level":"ERROR","path":"/v1/book","status":"500","duration":"231.876543ms","method":"POST"}